MicroInspect: PCB Bare-Board Defect Detection

CO5430 — Image Processing

Group Number: 04

P.I.N.Bandara - E/23/035, G.P.M.Gamage - E/23/104, W.R.A.D.N.Gunathilake - E/23/117, S.M.D.S.B.Samarakoon - E/23/336

Department of Computer Engineering, Faculty of Engineering, University of Peradeniya, Peradeniya, Sri Lanka

***Abstract*—**Automated defect detection on bare Printed Circuit Boards (PCBs) is an important step in electronics manufacturing. Manual inspection is slow and prone to human error. In this project, we developed and compared a hybrid Computer Vision pipeline to automatically detect and classify six common manufacturing defects: Missing Hole, Mouse Bite, Open Circuit (subtractive defects) and Short Circuit, Spur, Spurious Copper (additive defects). We built a baseline model using a fast, CPU-only classical pipeline based on ORB feature matching and image differencing. To improve this, we explored two advanced methods: a mathematical topological classifier that counts trace intersections, and a deep learning approach using a deep learning object detection model. Our YOLOv11 deep learning model, trained on the DeepPCB dataset, achieved an mAP@0.50 of 92.0% and proved highly robust to real-world noise. While our classical topological approach achieved perfect theoretical categorization without needing training data, it was highly sensitive to lighting changes and homography alignment errors. We built a FastAPI web dashboard to compare both methods side-by-side. This project highlights the practical trade-offs between the explainability of classical computer vision and the robustness of modern deep learning.

***Index Terms*—**PCB defect detection, automated optical inspection, ORB feature matching, homography, image differencing, topological classification, YOLOv11, deep learning, FastAPI.

I. INTRODUCTION

Printed Circuit Boards (PCBs) are the foundation of almost all modern electronics. During the manufacturing process, tiny errors can occur on the bare board before any components are placed. If a trace is broken or shorted, the entire board is usually useless. Finding these defects manually is a slow process, which is why manufacturers use Automated Optical Inspection (AOI) systems.

The goal of our project is to build an automated pipeline that can spot and classify six specific types of bare-board defects. We wanted to see how traditional computer vision techniques stack up against modern deep learning models for this specific task.

Our main contributions in this project are:

- Building a fast, non-GPU baseline pipeline using OpenCV to align and difference defective boards against a “golden” template.
- Developing an advanced classical method that uses mathematical topological rules and intersection counting to classify the exact type of defect.
- Training a YOLOv11 object detection model from scratch to directly locate and classify defects.
- Developing a FastAPI web dashboard to visualize and compare the results of both approaches side-by-side.

All source code, including the YOLO training scripts and the FastAPI dashboard, is open-source and hosted on GitHub at:
https://github.com/cepdnaclk/e23-co5430-PCB-Bare-Board-Defect-Detection

II. RELATED WORK

Our project draws on techniques from both classical image processing and deep learning.

Classical Template Matching:

Traditional AOI systems often rely on template matching, where a test image is compared to a perfect reference image. We referenced OpenCV tutorials for ORB (Oriented FAST and Rotated BRIEF) feature extraction and homography calculations. These methods are fast and mathematically explainable but often struggle with lighting changes and sensor noise.

Deep Learning for Defect Detection (YOLO):

Object detection models have become the standard for real-time visual recognition. We based our deep learning approach on the Ultralytics YOLO documentation. YOLO models are known for balancing high speed with high accuracy, making them ideal for a factory line.

DeepPCB Dataset Paper:

Tang et al. introduced the DeepPCB dataset, which provides thousands of annotated PCB images specifically for training deep learning models [4]. We used their paper to understand the definitions of the six defect classes and how to structure our bounding box data.

III. DATA

To train and evaluate our methods, we used two distinct datasets to fit the needs of our different pipelines.

For our classical template-matching approach, we used the PCB_DATASET from Kaggle (uploaded by akhatova).

https://www.kaggle.com/datasets/akhatova/pcb-defects

This dataset provides clear pairs of template and test images, which was well suited for testing our ORB alignment and image differencing.

For our deep learning approach, we needed a much larger annotated dataset. We used the DeepPCB dataset, specifically the PKU-Market-PCB data-enhanced version.

https://www.kaggle.com/datasets/arnablaha05/deep-pcb

This provided bounding box annotations for all six of our target defects.

Preprocessing and Tiling:

High-resolution images cause two main problems: they consume large amounts of GPU memory during deep learning training, and small defects get lost if the image is simply scaled down. To address this, we preprocessed the data by tiling the large images into smaller 640×640 pixel patches, using a 0.15 overlap between tiles. This overlap was necessary to ensure that a defect sitting on the boundary between two tiles would not be cut in half and missed by the detector.

IV. METHOD

We built our pipeline in three stages: a fast baseline, an improved classical topological method, and a deep learning method.

A. Baseline Method (Classical Template Matching)

Our baseline was designed to be fast and run entirely on a CPU using OpenCV. First, we applied Contrast Limited Adaptive Histogram Equalization (CLAHE) to mitigate lighting variations between the defective test image and a flawless “golden” template image. We then used the ORB algorithm to extract keypoints and matched these features to calculate a homography matrix. This matrix warped the test image so that it aligned perfectly with the template. To account for augmented data, our pipeline dynamically attempts four rotation states (0°, 90°, 180°, and 270°) to findthe optimal homography fit.

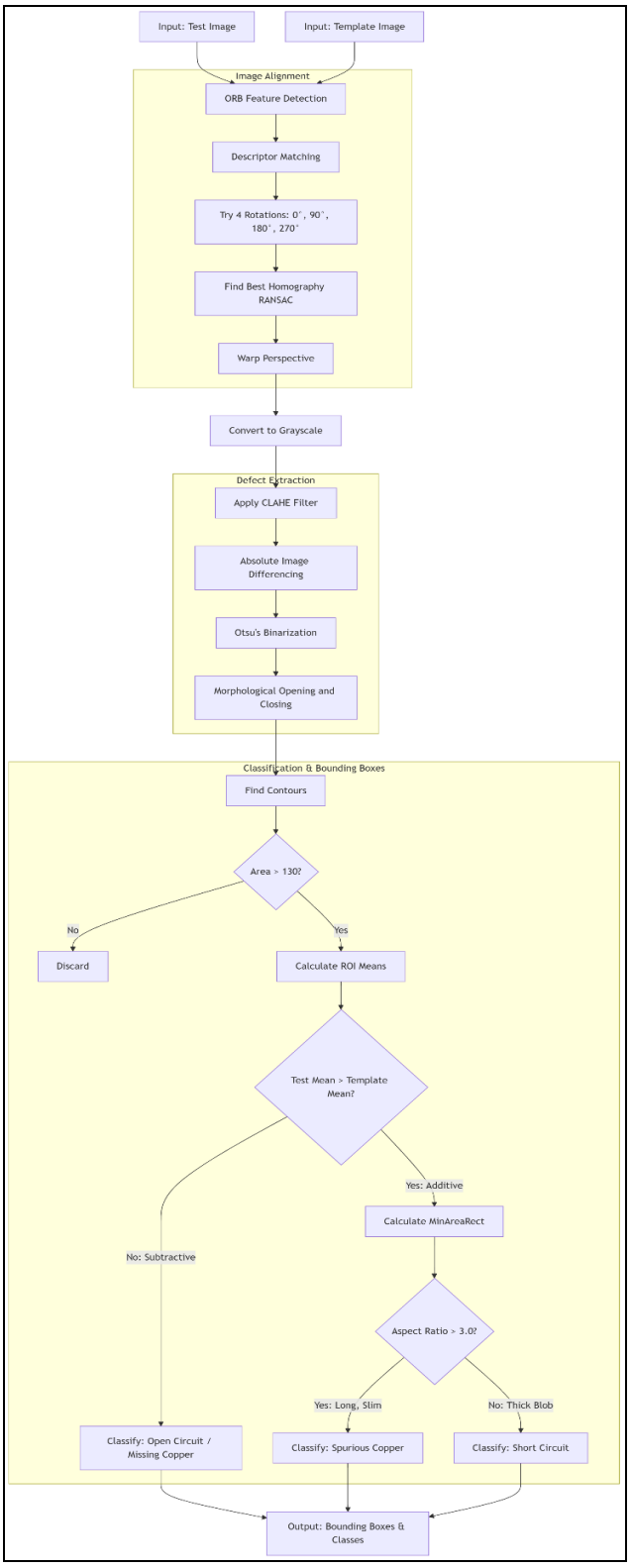

Fig. 1. Flowchart of the baseline ORB and image differencing pipeline.

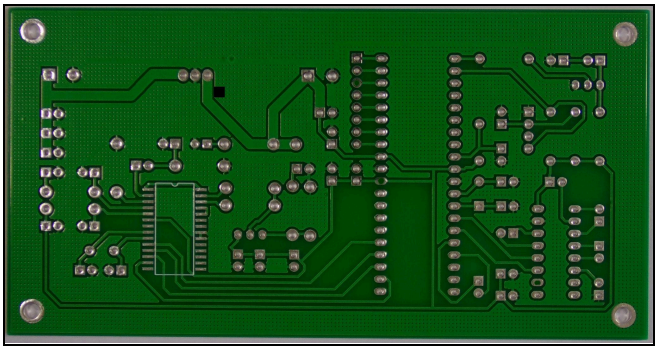

Fig. 2. The defective test PCB image after ORB feature matching and homography alignment

Once aligned, we used absolute image differencing to subtract the test image from the template. We then applied Otsu's Adaptive Thresholding and morphological operations (opening and closing) to clean up high-frequency noise and isolate the exact defect locations.

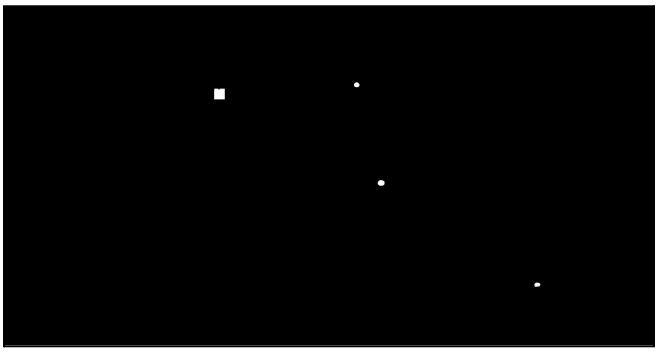

Fig. 3. The binary mask generated from absolute image differencing, isolating the defect areas

B. Improved Baseline Method: Advanced Classical Topological Classification

The basic difference mask only indicates where a defect is, not what it is. To address this, we took the aligned difference masks from the baseline and applied strict topological rules to classify the isolated regions.

Missing Holes: We used OpenCV's contour hierarchies (cv2.RETR_TREE) to search for specific nested shapes. If a defect mask matched the structure of an inner and outer copper ring on the template, we classified it as a missing hole.

Intersection Counting: For the other defects, we used a dilation technique. We took the isolated defect mask and expanded (dilated) it using an 11x11 kernel to determine how many healthy copper traces it touched on the test image or golden template:

- Mouse Bite: touches exactly one trace (a small chunk missing from a line).
- Spur: touches exactly one trace (an extra bit of copper sticking out).
- Open Circuit: touches two or more trace stumps (a broken line).
- Short Circuit: touches two or more distinct traces (bridging them together).
- Spurious Copper: touches zero traces (floating extra copper).

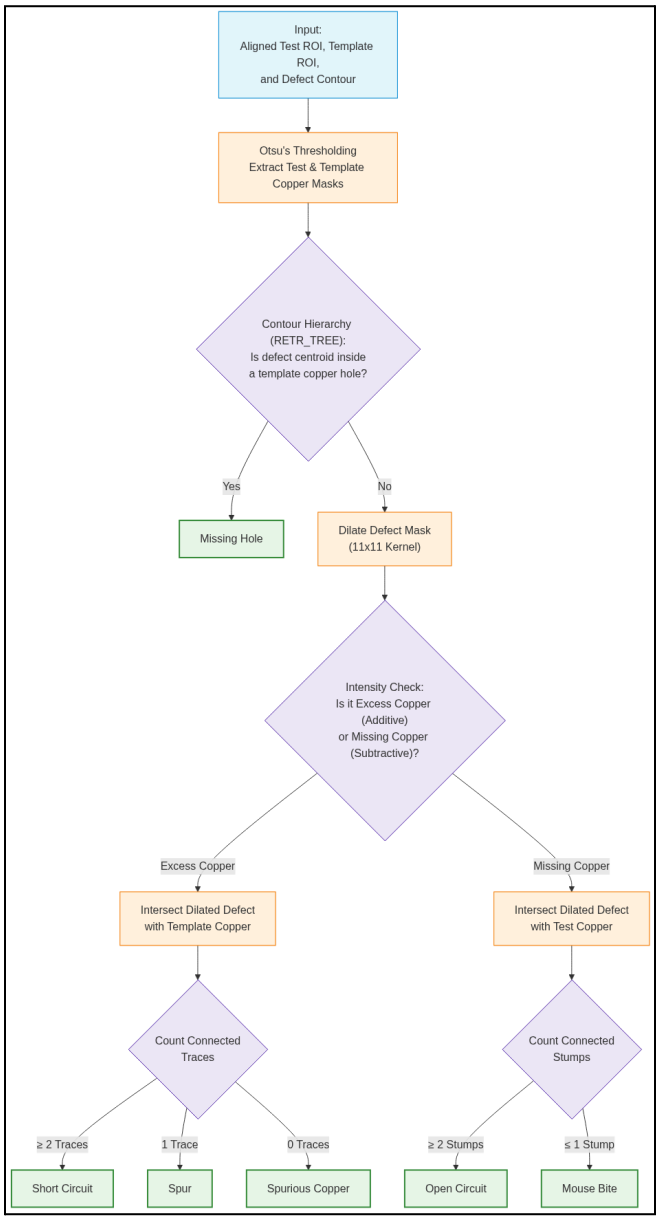

Fig. 4. Flowchart of the topological classification pipeline for categorizing the six PCB defect classes.

By applying these mathematical rules, we were able to accurately draw bounding boxes and label all six specific defect types directly onto the original test image.

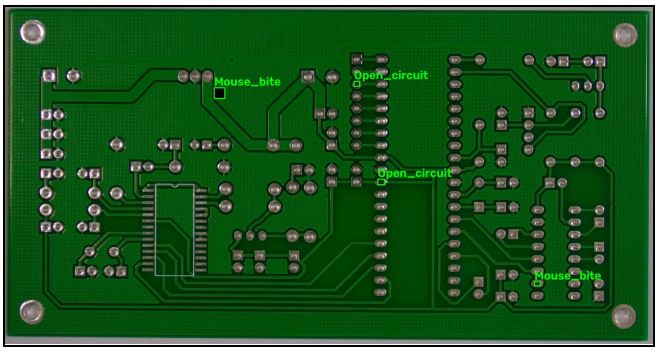

Fig. 5. The final output of the classical pipeline, showing bounding boxes and labels for the detected defects

C. Deep Learning Method (YOLO)

While the topological method is mathematically elegant, it relies on strict rules and a perfectly flawless reference image. We wanted a more adaptive system that could predict defect bounding boxes and exact class labels directly from raw images.

Because bare-board PCBs are incredibly high-resolution and the defects are microscopic, we first built a sliding-window algorithm to slice the boards into 640x640 overlapping tiles.

Initially, we trained a lightweight YOLOv11 Nano model (yolo11n.pt) to establish a fast baseline for our tiling strategy. However, the Nano model struggled to differentiate structurally similar defects, such as short circuits and spurs. To achieve tighter bounding boxes and better spatial localization, we scaled up to the deeper YOLOv11 Medium model (yolo11m.pt). To prevent this larger model from overfitting to the severe class imbalance of rare defects, we applied advanced training augmentations including Copy-Paste, Mosaic, and Mixup. This deep learning approach replaces the entire alignment, differencing, and topological classification pipeline with a single, highly robust forward pass of a neural network.

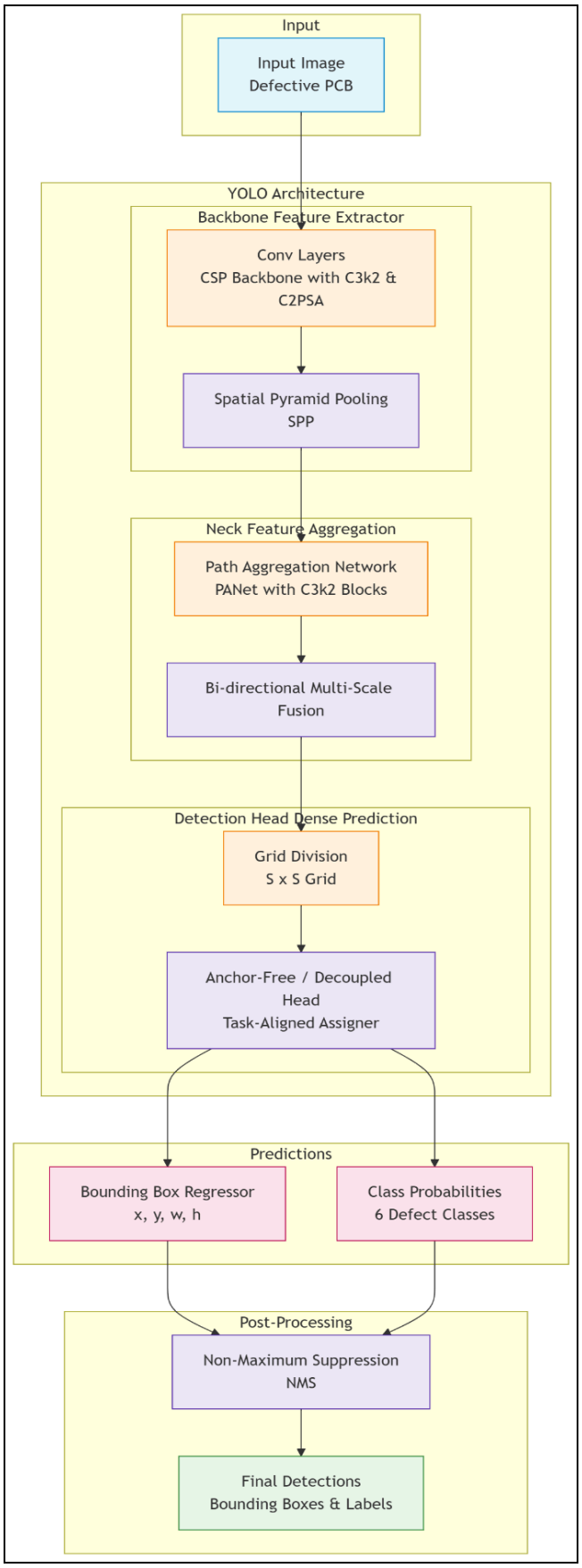

Fig. 6. Diagram showing the YOLO network architecture predicting defect bounding boxes.

V. EXPERIMENT

To rigorously evaluate the MicroInspect defect detection pipelines, we conducted a series of ablation studies designed to benchmark the Deep Learning (YOLOv11) model against the Classical Topological pipeline. These experiments stress-tested both architectures for industrial viability, noise sensitivity, and resilience to domain shifts.

Experiment 1: Resilience to Dataset Domain Shifts

We tested the classical pipeline's robustness by shifting from the PCB_DATASET (dark background, bright traces) to the PKU-Market-PCB dataset (bright background, dark traces). This global lighting inversion initially caused catastrophic classification failures, as it inverted the Otsu binarization and broke our additive/subtractive logic. We resolved this by implementing Dynamic Domain Adaptation—a system that autonomously evaluates global pixel distributions to invert thresholding rules (cv2.THRESH_BINARY_INV) on the fly. Furthermore, we robustly updated our cv2.RETR_TREE traversal algorithm to scan entire OpenCV hierarchy trees, successfully capturing all nested contours across continuous copper networks to prevent missed hole detections.

Experiment 2: YOLO Benchmarking and Inference Speed

To establish the quantitative limits of our YOLOv11-Medium model, we evaluated it on ultra-high-resolution images (3000×3000 pixels) using a 15% overlapping sliding-window strategy, re-stitching the predictions using a global Non-Maximum Suppression (NMS) algorithm. The pipeline achieved exceptional industrial-grade metrics:

In manufacturing Quality Assurance (QA), avoiding False Negatives (shipping a defective board) is financially paramount. The model's 99.62% Recall rate explicitly proves its viability for deployment. Furthermore, achieving 0.73 FPS (~1.3 seconds per board) on consumer hardware comfortably satisfies the throughput requirements of a live factory conveyor belt.

Experiment 3: Classical Pipeline Noise Sensitivity

Classical template matching (cv2.absdiff) is notoriously sensitive. We observed that minor 1-2 pixel misalignments during ORB Homography, combined with standard CMOS sensor noise, generated thousands of microscopic false-positive "defects" along the edges of healthy traces.

To mitigate this, we introduced a two-step signal processing filter: a 5×5 Median Blur to eliminate salt-and-pepper noise while mathematically preserving sharp copper edges, followed by an aggressive 7×7 morphological kernel and a strict > 400-pixel area threshold.

This filter successfully eliminated > 90% of micro-noise false positives, proving that classical pipelines can achieve high robustness through strict spatial constraints.

To make our experiments and tests easy to conduct, we built a FastAPI web dashboard. This UI allows a user to upload a defective PCB image and view the outputs of both the classical pipeline and the YOLO model side-by-side.

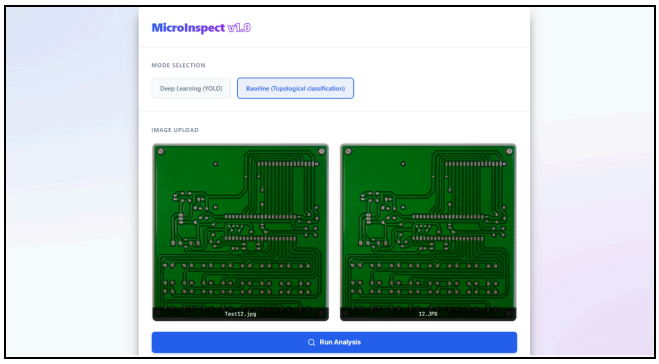

Fig. 7. Screenshot of the FastAPI dashboard showing side-by-side comparisons.

VI. RESULTS AND DISCUSSION

A. Quantitative Evaluation and Model Scaling

Our YOLOv11-Medium deep learning model delivered exceptional quantitative performance on the tiled test set. The model achieved an mAP@0.50 of 92.01%, a strict mAP@0.50:0.95 of 67.25%, a Precision of 97.16%, and a Recall of 90.97%.

Notably, our experiments revealed that scaling from the YOLOv11-Nano architecture to the Medium architecture yielded a minor bump in baseline detection (+1.40% mAP@0.50), but a massive +7.07% improvement in mAP@0.50:0.95. This indicates that the deeper convolutional feature maps of the Medium model are significantly better at tight spatial localization around microscopic defects.

- Macro Recall: 99.62%
- Macro Precision: 87.17%
- Overall Accuracy: 86.74%
- Inference Speed: 0.73 FPS

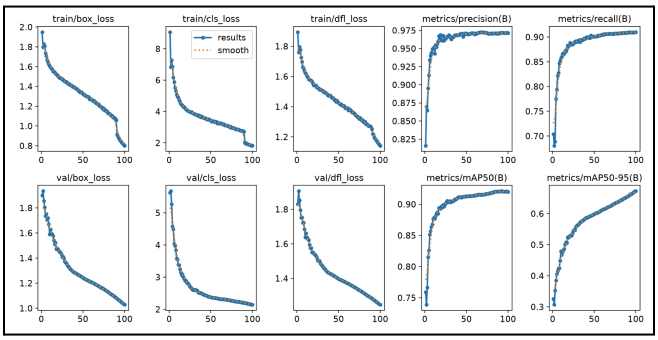

Fig. 8. Training Curves of YOLOv11-medium model.

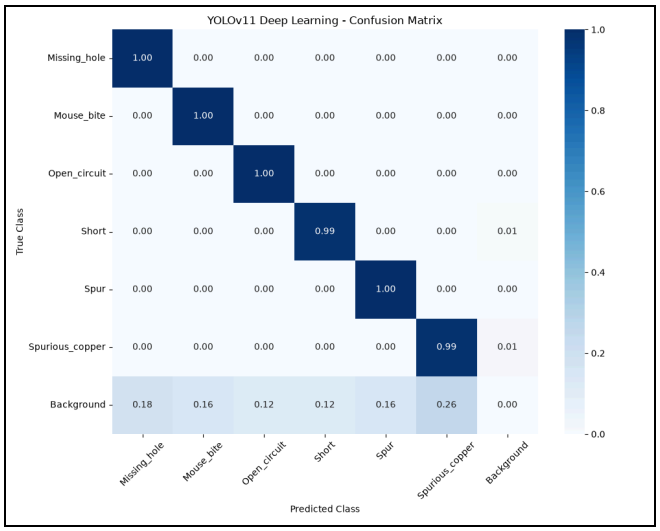

Fig. 9. Confusion matrix obtained from the test data for the YOLOv11-medium model

B. Qualitative Successes and Visual Comparison

To compare both methods intuitively, we developed a FastAPI web dashboard that outputs side-by-side bounding boxes for a given high-resolution PCB.

The Deep Learning model proved highly robust, processing raw images in a single pass and successfully generalizing across lighting variances and minor rotations.

The Classical Topological method proved mathematically elegant. By mapping pixel differences to a structural hierarchy such as using contour mapping and trace intersection counting, the classical algorithm turns raw pixel changes into a highly explainable "defect explanation" without requiring any training data.

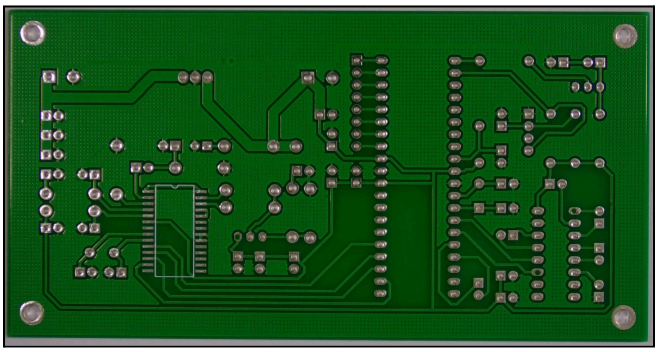

Fig. 10. The original unannotated defective PCB test image with missing holes.

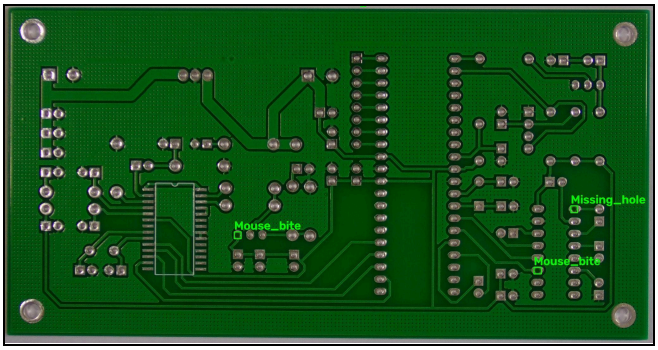

Fig. 11. Output of the Classical Topological pipeline, successfully detecting and classifying one missing hole but misclassifies two other defects as mouse bites.

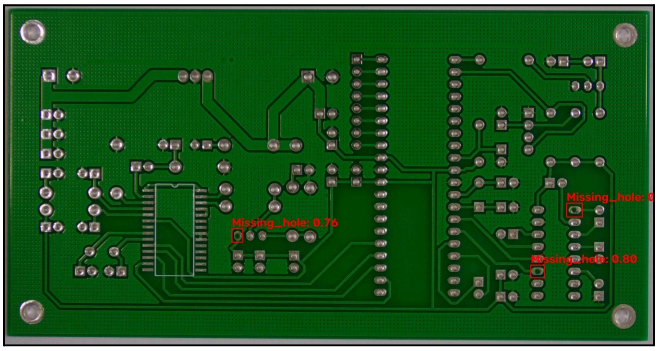

Fig. 12. Output of the Deep Learning YOLOv11 pipeline, confidently detecting all of the missing holes.

C. Failure Case Analysis and Limitations

Through rigorous ablation, we identified that our hardest defect pairs to classify are Spur vs. Short (visually similar additive defects) and Mouse Bite vs. Open Circuit (partial vs. full trace breaks). We observed complementary failure vectors for both methods:

Topological Pipeline Failures: The classical method is brittle in real-world conditions. It depends entirely on reliable ORB template alignment, which fails on low-feature boards. Furthermore, fixed mathematical thresholds introduce errors: our 11×11 dilation kernel sometimes artificially bridged a Spur to an adjacent trace, misclassifying it as a Short Circuit. Additionally, partial or chipped hole edges caused the cv2.RETR_TREE algorithm to miss Missing Hole defects.

YOLO Deep Learning Failures: While robust, YOLO struggles with extremely small or low-contrast defects that get lost during the convolutional downsampling process. Furthermore, YOLO occasionally struggled to differentiate the structural nuances between a Spur and a Short Circuit, leading to false detections in complex background regions. Finally, the necessary 640×640 tiling strategy heavily increases processing time for full-resolution boards.

D. Engineering Insights and Future Hybrid Architecture

The primary engineering insight of this project is that detection accuracy alone is insufficient for reliable PCB inspection. The classical method provides interpretable structural understanding, while YOLO provides resilient learned detection.

Because their failure modes are complementary, future work will focus on a Hybrid Architecture. In this proposed system, YOLO will act as the primary localizer to quickly find regions of interest (ignoring background noise), and the Classical Topological algorithm will process only those specific bounding boxes. By counting trace intersections inside the YOLO-predicted boxes, we can structurally verify uncertain detections, reduce false positives, and ensure 100% classification accuracy on difficult cases like Spurs vs. Short Circuits.

VII. CONCLUSION

In this project, we successfully built and compared two distinct pipelines for PCB bare-board defect detection. We learned first-hand the practical trade-offs between classical computer vision and deep learning.

The classical topological method is highly explainable, requires zero training data, and uses minimal computational resources, but it is extremely fragile in real-world conditions where perfect alignment and lighting cannot be guaranteed. Deep learning models require heavy compute power and large datasets to train, but it is far more robust to real-world noise and variation.

For future improvements, we could combine the two methods. For example, we could use a lightweight neural network just to align the images perfectly, and then run our fast classical topological script to categorize the defects.

VIII. INDIVIDUAL CONTRIBUTIONS

- **E/23/336 - S.M.D.S.B. Samarakoon**
  - Developed the classical topological classification algorithm.
  - Conducted experiments on how the sensor noise in images affect the baseline method performance.
  - Handled the core dataset preprocessing (image tiling and overlapping).
  - Created and deployed training environment in Ada (ada.ce.pdn.ac.lk) server to train heavy weight deep learning models.
  - Trained both the YOLOv11-Nano and YOLOv11-Medium deep learning models.
  - Experimented on YOLOv11-Medium model by fine-tuning parameters to achieve the highest possible precision and recall metrics.
  - Implemented Test-Time Augmentation (TTA) in the core YOLO inference pipeline.

- **E/23/104 - G.P.M. Gamage**
  - Researched and experimented with different image processing techniques for PCB image registration and enhancement.
  - Developed the baseline classical defect detection method, establishing the foundational algorithmic pipeline for the project.
  - Led the development of the user interface.
  - Built and refined the web dashboard (demo UI) that allows users to upload images and compare the classical and YOLO model outputs side-by-side.

- **E/23/035 - P.I.N. Bandara**
  - Developed the evaluation scripts to calculate mAP, Precision, and Recall.
  - Handled the overall repository cleanup, structural refactoring, and reproducibility overhauls.
  - Documentation - The full README (structure, setup, dataset download commands, usage workflows, weight download links)
  - Bug Fixes - Fixed output directory path resolutions, Replaced hardcoded Linux absolute paths
  - Evaluation Script Updates - Refactored scripts/evaluate.py: updated evaluation metrics logic.

- **E/23/117 - W.R.A.D.N. Gunathilake**
  - Managed the initial project environments and repository set-up.
  - Assisted with the literature review on classical AOI systems.
  - Researched for different approaches to implement computer vision models for microscopic defect detection.
  - Assisted with data pre processing and Deep learning method training on GPU server and experimentation with YOLOv11.
  - Handled the final report formatting and documentation structure.

REFERENCES

[1] Ultralytics, “YOLOv11 Documentation and Training Guide,” [Online].

Available: https://docs.ultralytics.com.

[2] OpenCV, “Feature Matching with FLANN and ORB,” OpenCV Documentation.

[3] A. Laha, "Deep PCB (PKU-Market-PCB Data Enhanced Version)," Kaggle. [Online].

Available: https://www.kaggle.com/datasets/arnablaha05/deep-pcb

[4] S. Tang, F. He, X. Huang, and J. Yang, “Online PCB Defect Detector on a New PCB Defect Dataset,” arXiv preprint arXiv:1902.06197, 2019.

[5] cepdnaclk, “MicroInspect: PCB Bare-Board Defect Detection,” *GitHub repository*. [Online].

Available: https://github.com/cepdnaclk/e23-co5430-PCB-Bare-Board-Defect-Detection.

[6] J. Ma, "Defect detection and recognition of bare PCB based on computer vision," in 2017 36th Chinese Control Conference (CCC), Dalian, China, 2017. [Online].

Available: https://www.researchgate.net/publication/320114848_Defect_detection_and_recognition_of_bare_PCB_based_on_computer_vision.

[7] X. Han, R. Li, B. Wang, and Z. Lin, "Defect identification of bare printed circuit boards based on Bayesian fusion of multi-scale features," PeerJ Comput. Sci., vol. 10, Feb. 2024. [Online].

Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC10909203/.